(Structure-Aware Chunking 기반 Legacy Code Dataset 포함)

> "이 연구는 RAG 시스템에서 청킹을 단순 전처리가 아니라
> 정보 재사용성을 결정하는 핵심 구조 요소로 재정의하고,
> 텍스트와 레거시 코드 도메인에서 그 효과를 실증한 연구입니다."

---

0. 연구 방향 한눈 요약 (Executive Summary)

본 연구는 **도메인 특화 RAG 시스템에서 청킹(Chunking)이 검색 정확도와 정보 재사용성에 미치는 구조적 영향을 분석**하고,

텍스트 및 코드 도메인 모두에서 **구조 인식 기반 청킹(Structure-Aware Chunking)** 이

기존 고정 길이 청킹 방식의 한계를 어떻게 극복할 수 있는지를 실증적으로 보이는 것을 목표로 한다.

특히,

- **텍스트 도메인:** Harry Potter 시리즈
- **코드 도메인:** Legacy Python Code

라는 **성격이 다른 두 데이터셋**을 동시에 다룸으로써,

청킹 문제가 특정 도메인에 국한된 문제가 아니라

RAG 시스템 전반의 구조적 설계 문제임을 보인다.

---

1. 연구 개요 (Overview)

RAG(Retrieval-Augmented Generation)는

LLM의 환각(hallucination) 문제를 완화하고,

외부 지식을 기반으로 한 신뢰 가능한 응답 생성을 가능하게 하는 핵심 아키텍처로 자리 잡았다.

그러나 최근의 RAG 시스템은 다음과 같은 **전처리 단계의 구조적 취약성**을 공통적으로 가진다.

- 문서를 일정 길이로 잘라내는 **Fixed-size Chunking**

- 혹은 줄 단위 기반의 단순 분할

이 방식은 구현은 간단하지만,

정보 검색의 최소 의미 단위와 실제 데이터의 구조적 단위가 불일치하는 문제를 야기한다.

본 연구는 이 문제를

단순한 하이퍼파라미터 튜닝(Chunk Size 조절)의 문제가 아니라,

- *“구조를 무시한 전처리 설계의 문제”**로 재정의한다.

---

2. 연구 배경 및 문제 정의 (Motivation & Problem Statement)

2.1 기존 RAG 시스템의 설계 가정

기존 RAG 시스템은 암묵적으로 다음과 같은 가정을 전제한다.

1. 텍스트는 선형적(linear)으로 읽힌다
2. 일정 길이의 청크는 독립적인 의미 단위로 기능한다
3. 청킹은 검색 성능에 큰 영향을 주지 않는 전처리 단계이다

그러나 실제 도메인 데이터는 이러한 가정을 충족하지 않는다.

2.2 텍스트 도메인에서의 문제: Harry Potter

Harry Potter 시리즈는 도메인 특화 RAG 실험에 자주 사용되는 데이터셋이며,

다음과 같은 특성을 가진다.

- 고유 명사(주문명, 인물명, 마법 설정)의 높은 밀도
- 서사적 문맥에 강하게 의존하는 의미 구조
- 챕터 및 장 단위의 상위 문맥 중요성

고정 길이 청킹을 적용할 경우:

- 고유 명사가 문맥 없이 분리됨
- 질문에 필요한 정보가 여러 청크에 흩어짐
- 검색 Recall은 유지되더라도 Precision이 급격히 하락

2.3 코드 도메인에서의 문제: Legacy Python Code

코드 데이터는 텍스트보다 문제를 더 명확히 드러낸다.

- 코드의 의미 단위는 **라인이 아니라 구조**
- `class`, `def`, 제어 블록은 하나의 의미 덩어리

라인 기반 혹은 토큰 기반 청킹은 다음을 초래한다.

- 함수 정의의 중간 절단
- 변수 스코프 및 제어 흐름 붕괴
- 실행 불가능한 코드 조각 생성
- **Orphan Code 발생**

👉 이는 LLM이 "이 코드가 무엇을 하는지"를 추론하는 것 자체를 불가능하게 만든다.

---

3. 핵심 문제 재정의: Chunking은 구조 문제다

3.1 기존 청킹 방식의 한계 요약

청킹 방식	핵심 한계
Fixed-size Chunking	Noise 증가, 문맥 단절
Line-based Chunking	구조 붕괴, 실행 불가
Token-based Chunking	의미 단위 손실

이로부터 도출되는 결론은 명확하다.

> **Chunking의 실패는 길이 조절의 실패가 아니라구조 인식의 실패이다.**

---

4. 제안 방법: Structure-Aware Chunking

4.1 설계 원칙

본 연구의 구조 인식 청킹은 다음 원칙을 따른다.

1. **의미 단위는 구조에서 나온다**
2. 청킹은 검색 정확도에 직접 영향을 미친다
3. 좋은 청크는 재사용 가능한 지식 단위여야 한다

4.2 텍스트 도메인 적용

- 기본 분할 단위: 문단 / 챕터

- 메타데이터(출처, 장 번호) 유지
- Chunk Size 축소 상황에서도 문맥 유지 여부를 중심으로 평가

4.3 코드 도메인 적용 (AST 기반)

- Python `ast`를 활용해 코드 구조 파싱
- 청킹 기준:
  - Class 단위
  - Function 단위
  - 제어 블록 단위
- 부모 구조 정보를 메타데이터로 주입

```
# [Parent: Class OrderService > def validate_order]
if order.amount > limit:
    raise ValueError("Exceed limit")
```

→ 독립 청크임에도 **문맥 복원 가능**

---

5. 제안 아키텍처 (Proposed System Architecture)

5.1 전체 파이프라인 관점

```
Raw Data
 ├─ Text (Harry Potter)
 └─ Code (Legacy Python)
        ↓
Structure-Aware Chunking
        ↓
Embedding
        ↓
Hybrid Retrieval (BM25 + FAISS)
        ↓
RRF Fusion
        ↓
LLM Generation
```

본 연구에서는 **청킹 단계를 단순 전처리가 아닌Information Reuse를 결정하는 핵심 모듈**로 정의한다.

---

6. 실험 설계 (Evaluation Methodology)

6.1 실험 목적

- Chunk Size 축소 상황에서의 Retrieval Robustness 검증
- 구조 인식 청킹이 정보 재사용성에 미치는 영향 분석

6.2 데이터셋 구성

구분	데이터셋	목적
Main	Harry Potter	도메인 특화 RAG 성능
Auxiliary	Legacy Python Code	구조 보존 효과 검증

6.3 평가 지표

Retrieval Metrics

- Hit Rate@K
- MRR

Structural Metrics (Code)

- Syntax Error Rate
- Orphan Code Ratio
- Executable Chunk Ratio

Reuse Metrics

- 동일 인덱스 기반 다중 태스크 수행 가능성

---

7. 예상 결과 및 분석 관점

- 작은 Chunk Size 환경에서:
  - 기존 방식: 성능 급락
  - 제안 방식: 성능 안정적 유지

- 코드 도메인에서 구조 인식 효과가 더욱 명확하게 드러날 것으로 예상

---

8. 연구 기여 정리 (Contributions)

학술적 기여

- Chunking을 **Information Reuse 관점의 설계 문제로 재정의**
- 구조 기반 RAG 설계의 도메인 일반성 제시

실무적 기여

- 레거시 코드 분석, 코드 QA, 문서화 자동화에 즉시 활용 가능
- RAG 전처리 설계 가이드라인 제공

---

9. IEEE IRI 관점 정합성

- Information Reuse: 재사용 가능한 지식 단위 설계
- Information Integration: Hybrid Retrieval + 구조 통합
- System-level Contribution: 파이프라인 수준 개선